Predicting Ligue 1 Match Outcomes with Player-Level Statistics

Data Collection & Feature Organisation

Sienna O'Shea, Samuel Majbruch, Ioana Olaru & Ava Sudit

Ligue 1 · Seasons 2018–2024 · 1,653 matches · 73 features

Outline

Central Hypothesis

Research Question

Do individual football player rolling statistics add predictive signals beyond team-level aggregates?

Goal: predict Ligue 1 match outcomes (home win, draw, away win) using team and player data. Train on 2018–2022; test on 2023/24.

Three feature regimes evaluated:

Regime A

Team-level aggregates

29 features

Regime B

Team + FBref match statistics

50 features

Regime C

Full set incl. player-rolling features

76 features

Data Sources — Three Pipelines

FBref

What: Team-level match records and player-season summaries

Seasons: 2018–2023

Rows: 62,683 player-match rows

Script: `collect_fbref.py`

Understat

What: Match results, xG values, and pre-match forecast probabilities

Seasons: 2018–2023

Rows: 2,105 matches

Script: `collect_understat.py`

Transfermarkt

What: Estimated squad market values (euros) and average player age

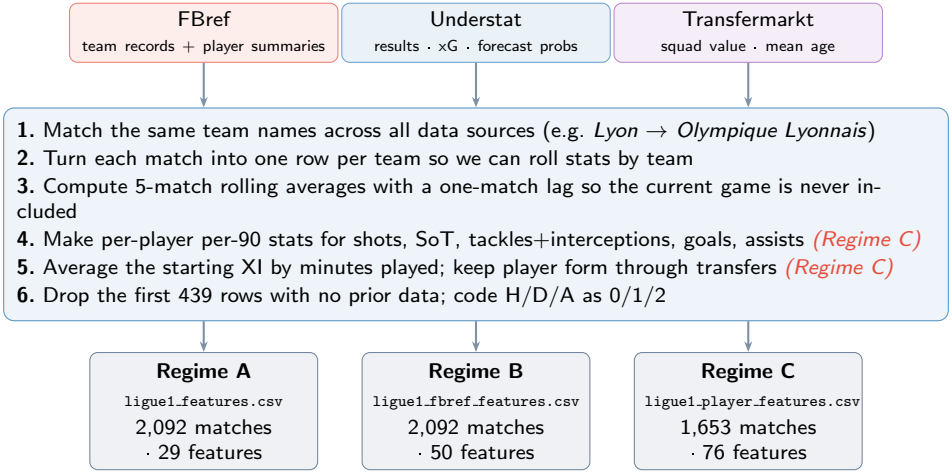
Seasons: 2018–2023

Rows: 174 team-seasons

Script: `collect_transfermarkt.py`

FBref data was extracted via a scripted pipeline; Understat data via its API; Transfermarkt values from a reliable GitHub Repo. The 2021/22 FBref season required manually completing a CAPTCHA before resuming the scripted download.

Data Pipeline



Dataset at a Glance

Feature regimes

Regime	Matches	Features
A: team only	2,092	29
B: +FBref	2,092	50
C: +player	1,653	76

Regime C loses 439 rows from cold-start roll-up.

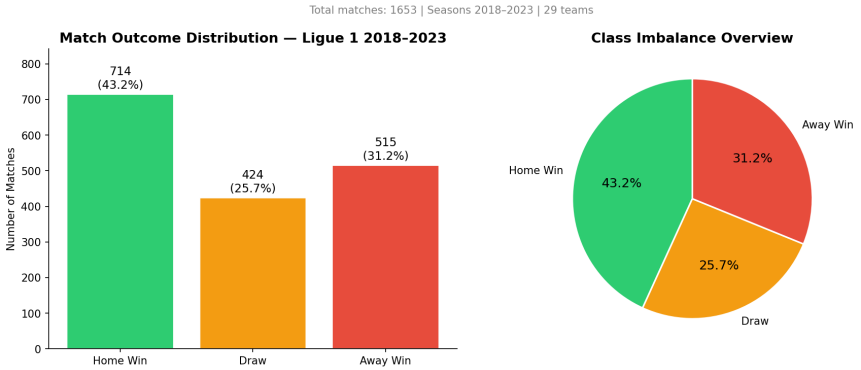
Features (Regime C)

Source	Features
Understat / Transfermarkt	29
FBref team-level	21
Player-level	26
Total	**76**

Season breakdown

Season	Matches	Note
2018/19	370	
2019/20	278	COVID curtailed
2020/21	379	
2021/22	379	CAPTCHA step
2022/23	380	
2023/24	306	Partial (May 2024)
Total	**2,092**	

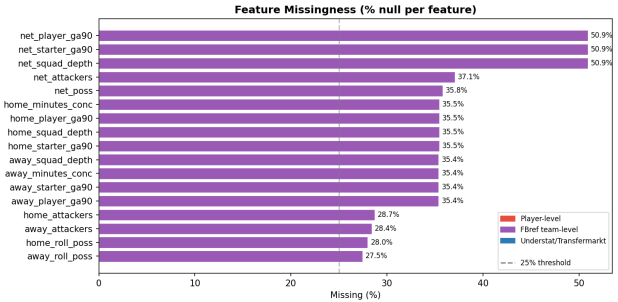
Plot 1 — Target Variable Distribution



Class imbalance

Draws are underrepresented at ≈26%. Models must use class weights or calibrated probabilities — accuracy alone is a misleading metric.

Plot 3 — Missing Data: Sources & Resolution



What is missing?

- No FBref *team* data for 2021/22 – not able to collect team-level data, but player-level was captured
- Rolling features need prior matches — first ≈5 matchdays per season are null (cold-start)
- `net_*` at ∼51%: null if *either* side null

What we did

- No imputation – null rows dropped at fit time
- Same subset used across all regimes
- Regime C: 439 cold-start rows dropped → 1,653 matches

What this means

- 2021/22 excluded equally from A, B, C – no regime penalised
- Regime C subset smaller – note when comparing across regimes
- Understat / Transfermarkt: zero missingness

Feature Engineering

Source	Features engineered	Regimes
Understat + Transfermarkt	Rolling xG, goals, wins, points; squad value, age	A, B, C
FBref team-level	Rolling possession, formation, goals + assists per 90 (team)	B, C
Player-level	Rolling shots, shots on target, defensive actions, goals, assists per 90	C only

- **Anti-leakage:** 5-match rolling window with `shift(1)` — current match never in its own features
- **Bookmaker probabilities excluded** from model inputs — kept as external baseline only (including them would be circular)
- **Player rolling stats are personal, not club-based** — if a player transfers, their form history carries over

Player-Level Aggregation

Each player's 5-match rolling stat is aggregated across the starting XI, **weighted by minutes played**:

$$\hat{s}_{\text{team}} = \frac{\sum_{p \in \text{XI}} \text{min}_p \cdot \text{roll}_p}{\sum_{p \in \text{XI}} \text{min}_p}$$

Each stat then becomes three features:

`home_pl_*` `away_pl_*` `net_pl_*`

Stats tracked per player

Feature	Meaning
`sh_p90`	shots / 90
`sot_p90`	shots on target / 90
`def_p90`	tackles + int. / 90
`gls_p90`	goals / 90
`ast_p90`	assists / 90
`starter_mins`	minutes regularity
`pct_fw`	fraction forwards
`depth_sh`	starter vs bench gap

Feature Selection — Four-Method Protocol

Each of the 73 features was scored by **four independent methods**, percentile-normalised (0–1), and averaged into a **composite score**.

1. Filter Method

Simple idea: if I know this feature, do I get a better clue about the match result? Good for catching patterns that are not just straight lines.

2. Wrapper Method

Start with many features, remove the weakest ones step by step, and keep the set that gives the best validation score. Best subset: **26 features**.

3. Embedded Method — Lasso Regression

A linear model with a penalty that pushes unhelpful features down to exactly zero. 17 features were removed; **0 player features** were.

4. Embedded Method — Decision Tree

Let a forest of trees rank which features it uses most to make good splits. This gives a second opinion that broadly matches RFECV.

Based on Lecture 9's taxonomy of feature selection methods: filter, wrapper, and embedded methods. Information gain is developed in Lecture 8; Lasso regularization is covered in Lecture 4; and decision-tree / random-forest feature importance is connected to Lecture 8.

Temporal CV (TimeSeriesSplit, 5 folds) throughout — no shuffled cross-validation.

Plot 4 — Feature Distributions by Match Outcome (Top 20)

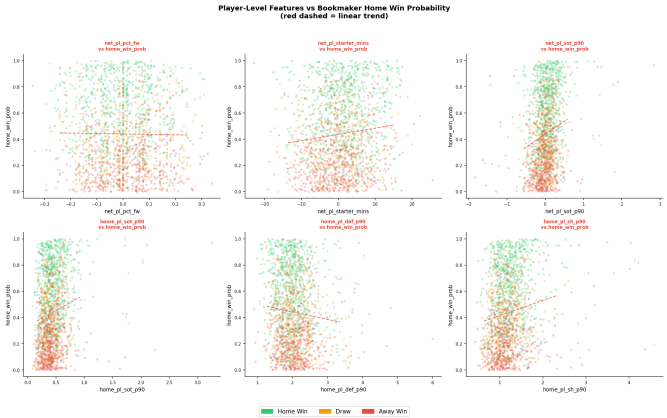
How to read: each panel shows the distribution of one feature split by match outcome (H/D/A).

Less overlap between the three coloured distributions ⇒ more discriminative feature.

Bookmaker probability features show the clearest separation.

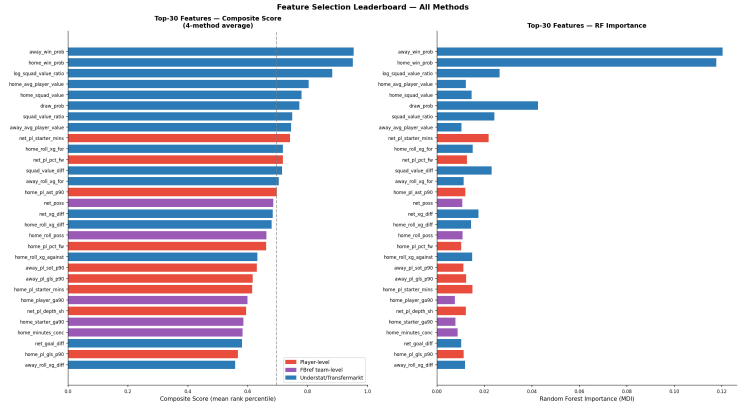
Player-level features add a small but consistent nudge on top of the bookmaker signal.

Plot 8 — Bookmaker Odds vs Player-Level Features



The bookmaker's implied probability and the player rolling stats **mostly disagree** — wide scatter, no tight trend. This is a good sign: player features are contributing *new information* that the bookmaker does not fully price in.

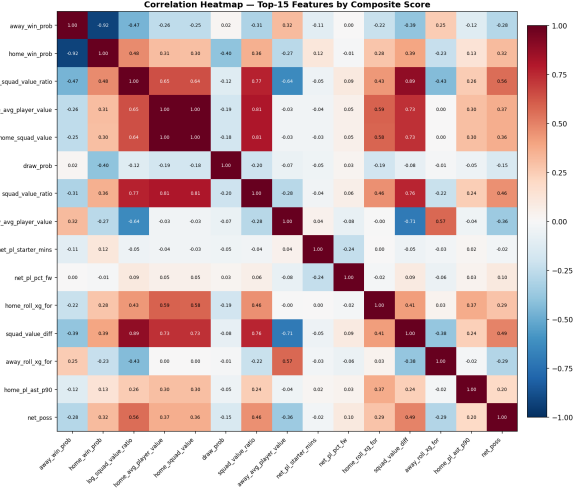
Plot 12 — Feature Leaderboard: All 73 Features Ranked



■ Understat/Transfermarkt ■ FBref team-level ■ Player-level

Two player features enter the top 10: `net_pl_starter_mins` (rank 9) and `net_pl_pct_fw` (rank 11). 10 player features survive RFECV wrapper selection.

Plot 14 — Correlation Heatmap: Top 15 Features



Red zones = high correlation ⇒ redundant (e.g. `home_win_prob` and `away_win_prob` are two sides of the same bet).

Player features (`net_pl_pct_fw`, `net_pl_starter_mins`) show *near-white rows and columns* — barely correlated with anything else.

This confirms they bring **genuinely new information** rather than repeating the bookmaker signal.

Modeling — Protocol

Three feature regimes compared on the **same** 593-match common subset (NaN-free across all three CSVs), with a strict temporal holdout: train on seasons 2018–2022 (425 matches), test on season 2023/24 (168 matches).

Models

Baseline: Understat pre-match win probabilities.

Linear: multinomial logistic regression (scaled, $C = 1$).

Trees: Random Forest, XGBoost, and a 50/50 **ensemble** of the two.

Tuning & calibration

Search: `RandomizedSearchCV` (100 configs) over `TimeSeriesSplit(5)`, scored by log-loss.

Imbalance: class weights (RF) / sample weights (XGB) to handle rare draws.

Calibration: isotonic via `CalibratedClassifierCV` on a temporal inner split.

Metrics: log-loss (primary, lower is better), Brier, accuracy. Uniform reference = 1.099, Understat baseline = 0.910.

Modeling — How we tune: RandomizedSearchCV + TimeSeriesSplit

Every model in this deck is tuned with the same protocol — designed to pick hyperparameters **without ever peeking at the future**.

TimeSeriesSplit(5) — fair scoring

Standard cross-validation shuffles rows randomly, which would let the model "see" May 2024 to predict November 2023.

`TimeSeriesSplit` keeps the validation window strictly *after* the training window:

- Fold 1: train [1–85] → validate [86–170]
- Fold 2: train [1–170] → validate [171–255]
- . . . (5 folds total)

Each fold simulates real deployment: "stand on date X,

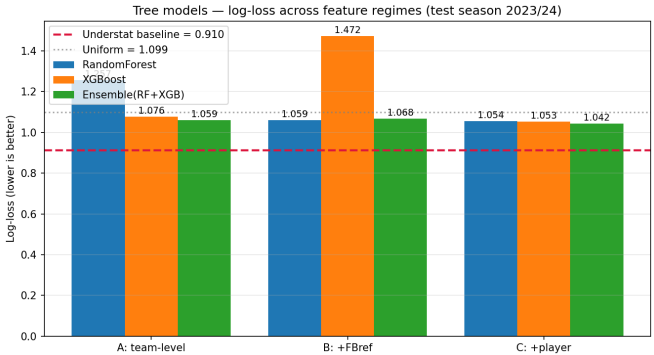
RandomizedSearchCV — efficient knob search

Models have hyperparameters (k, max_depth, learning_rate. . .). XGBoost alone has ∼17,000 combinations on our grid — too many to try exhaustively.

`RandomizedSearchCV` samples `n_iter` random configs (we used 30–100), scores each across the 5 time-respecting folds, and refits the winner on the full training set.

Total work per model: 30 configs × 5 folds = 150 fits, all leak-free, then 1 final refit on the held-out test season.

Modeling — Results



Test log-loss (season 2023/24)

	A	B	C
RF	1.26	1.06	1.05
XGB	1.08	1.47	1.05
Ensemble	1.06	1.07	**1.04**
LR (ref)	1.06	1.14	1.22

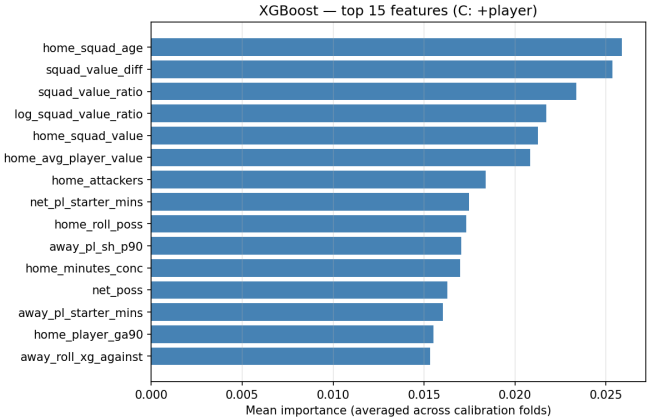
A=*team*, **B**=*+FBref*, **C**=*+player*

Accuracy in context (3-class):

Random guess	33%
Always home	43%
Best model	50%
Bookmaker	**56%**

No model clears the bookmaker baseline (0.91 log-loss). Bookmakers have real-time info (injuries, team news) that rolling historical stats cannot capture.

Modeling — Feature Importance (XGBoost, full regime)



How to read: mean importance over the calibration folds for the best XGBoost on the full player regime.

Team-level form (rolling xG, market value) sits at the top, as expected.

Player-level features (`starter_mins`, `pct_fw`, depth gaps) appear repeatedly in the top ranks — they are not just noise the model tolerates, they are signal the model actively uses.

Consistent with the feature-selection leaderboard (`net_pl_starter_mins` rank 9, `net_pl_pct_fw` rank 11).

Modeling — What changed vs Logistic Regression

1. Tree models flip the feature-regime story

Logistic regression *got worse* as features were added ($1.06 \rightarrow 1.14 \rightarrow 1.22$): multicollinearity and curse of dimensionality. Tree models *improve* with more features (RF: $1.26 \rightarrow 1.06 \rightarrow 1.05$) — they handle correlated inputs natively.

2. Player features add real signal

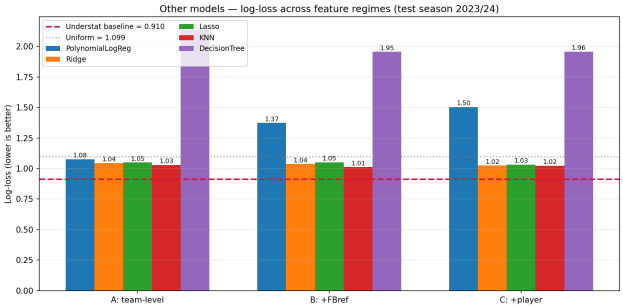
The full player regime (C) is the best for every tree model. Confirms the project hypothesis: player-level rolling stats add predictive power once the model can use them.

3. Ensembling extracts a final gain

RF and XGB make different mistakes; averaging their calibrated probabilities beats either alone (1.04 vs 1.05/1.05 in regime C).

Other Models — Linear, Polynomial, KNN, Decision Tree

Same protocol, same temporal holdout, same 3 regimes. We tested four additional model families to see whether the choice of algorithm actually matters at our sample size.



Test log-loss (season 2023/24)

	A	B	C
Polynomial	1.08	1.37	1.50
Ridge (L2)	1.04	1.04	**1.02**
Lasso (L1)	1.05	1.05	1.03
KNN	1.03	**1.01**	1.02
Decision Tree	2.13	1.96	1.96
RF+XGB ens.	1.06	1.07	1.04

A=*team*, **B**=*+FBref*, **C**=*+player*

Six algorithms cluster within 0.03 log-loss — we've hit the *information ceiling* of post-match aggregate stats. Closing the gap to Understat (0.91) needs richer features, not a different model.

Per-model takeaways: Polynomial overfits as features grow (d=2 explodes to 400^+ inputs); **Ridge/Lasso** match each other — regularisation type doesn't matter much here; **KNN** surprises by tying the tree ensemble (small dataset → simple wins); **Decision Tree** confirms the value of bagging/boosting — a single unregularised tree is the worst model in every regime.

KNN — Choosing k

k was not hand-picked. The full grid $k \in \{5, 10, 15, 20, 30, 50\} \times$ {uniform, distance} $\times$ {Manhattan, Euclidean} was searched under the same `TimeSeriesSplit(5)` protocol and the lowest-log-loss combination was kept.

Why those bounds?

MIN $k = 5$: smaller k gives degenerate probabilities (denominators of 2–3 only output $\{0, 0.33, 0.5, 0.66, 1\}$) — log-loss punishes that hard.

MAX $k = 50$: the smallest CV training fold is ∼75 rows; sklearn requires $k \leq$ fold size. We initially tried $k = 100$ and got runtime warnings, so we capped at 50.

Winners (per regime)

Regime	k	metric
A: team	50	Euclidean (p=2)
B: +FBref	50	Manhattan (p=1)
C: +player	50	Manhattan (p=1)

All three regimes **independently** picked $k = 50$ with distance-weighting. The Minkowski exponent drifted from p=2 to p=1 as features grew — consistent with the textbook result that L1 is more robust in higher dimensions.

Why $k = 50$ makes domain sense: distance-weighting means the closest 5–10 historical matchups still drive the prediction; the further 40 just provide a regularising prior. Football is noisy, so averaging ∼12% of the historical sample smooths randomness without washing out the matchup-specific signal.

Key Findings

1. Player-level rolling statistics add real predictive signal beyond team form and bookmaker odds.
2. Feature selection backs that up: `net_pl_starter_mins` and `net_pl_pct_fw` rank highly across all four methods, and RFECV keeps 10 player features in its final subset.
3. Tree models give the clearest proof of the player signal: as we move from team-only features to the full player regime, every tree model improves, while logistic regression gets worse under the extra correlation and dimensionality.
4. The best tree result is the RF + XGBoost ensemble on the full player regime at **1.04** log-loss and 50.6% accuracy, so ensembling still extracts a small final gain.
5. KNN is a useful surprise: with k=50 it ties the stronger models (**1.01** in regime B, 1.02 in regime C), suggesting that averaging similar past match states works well when the sample is modest and noisy.
6. Six different algorithms finish within about 0.03 log-loss of one another, which points to